

TSAC: Transformer-based Soft Actor-Critic for Behavior Decision-making in Autonomous Driving

1st Zihan Gao
Shenyang University of Technology
Shenyang, China
zhgao@smail.sut.edu.cn

2nd Shixian Wang*
Shenyang University of Technology
Shenyang, China
sxwang@sut.edu.cn

3rd Zhijia Zhang
Shenyang University of Technology
Shenyang, China
zhangzj@sut.edu.cn

Abstract—Traditional SAC relies on CNNs for processing visual inputs, which may fail to capture global context crucial for urban navigation. To address this issue, this work explores the integration of Transformer blocks into Soft Actor-Critic (SAC) algorithm for autonomous driving behavior decision-making. Transformers, with their self-attention mechanisms, excel at modeling long-range interactions and global features, potentially enhancing SAC's policy learning. We propose a Transformer-based SAC architecture to improve the agent's understanding of complex driving scenes, focusing on critical regions and interactions. This approach aims to achieve more accurate value estimates, better policy gradients, faster convergence, and safer driving policies. The evaluation against CNN-based SAC demonstrates improved convergence speed and decision-making performance in autonomous driving scenarios.

Keywords—Autonomous Driving; Behavior Decision-making; SAC; Transformer

I. INTRODUCTION

Autonomous driving decision-making is a complex problem that requires processing high-dimensional sensory inputs and making safe, real-time decisions in dynamic environments. Reinforcement learning (RL) has emerged as a promising approach for training autonomous driving policies in simulation, allowing an agent to learn by trial-and-error how to navigate traffic scenarios. In particular, the Soft Actor-Critic (SAC) algorithm has gained attention in this domain due to its strong performance in continuous control tasks and its stability from entropy regularization. SAC's ability to learn stochastic policies with off-policy data has been leveraged in driving simulators (e.g. CARLA) to train vehicles that can safely navigate intersections and roundabouts, achieving high success rates and low collision rates compared to alternative methods like DQN or PPO.

However, a limitation in many SAC-based autonomous driving systems is the reliance on convolutional neural networks (CNNs) for processing visual observations. CNNs are effective at extracting local features from images, but they have a limited receptive field and may struggle to capture global context or long-range interactions between objects in a scene. In a crowded urban setting, understanding the relationships between distant vehicles, pedestrians, and traffic

signals is crucial for good decision-making. Recent advances in deep learning suggest that transformer models, which use self-attention mechanisms, can capture such relationships more effectively. Originally popularized in natural language processing, transformers have demonstrated impressive results in vision tasks by modeling an image as a sequence of patches and attending to global features. Vision Transformers (ViT) have even outperformed CNNs on large-scale image recognition, indicating their high capacity to learn global representations.

By integrating a transformer into the SAC architecture, we aim to enhance the agent's understanding of complex driving scenes. The transformer's self-attention can enable SAC's network to focus on critical regions (such as an abrupt pedestrian crossing or a distant speeding car) and model interactions between the ego-vehicle and other agents. This richer scene representation could lead to more accurate value estimates, better policy gradients for the Actor, faster convergence, and ultimately safer driving policies. In this paper, we propose a Transformer-based SAC and demonstrate its benefits in autonomous driving scenarios. We provide a thorough evaluation against the traditional CNN-based SAC to show improved convergence speed and decision-making performance. The remainder of this paper is organized as follows: Section II reviews related works, Section III details the methodology and architecture, Section IV presents experimental results, Section V concludes with future directions.

II. RELATED WORKS

A. Reinforcement Learning for Autonomous Driving

Deep RL has been applied to autonomous driving decision-making with encouraging results. SAC, in particular, has been used to train vehicles in simulators to handle complex maneuvers. For example, Elallid *et al.* used SAC to guide an autonomous vehicle through roundabouts safely, using a CNN to process front camera images and a goal vector; their SAC agent outperformed DQN and PPO baselines in success rate and collision avoidance [1]. Savari *et al.* integrated SAC into a lane-changing scenario and found it improved driving efficiency compared to rule-based controllers [2]. These

studies highlight SAC's strengths (stability from entropy maximization and off-policy efficiency) for driving tasks. However, they generally rely on CNN feature extractors and

sensor inputs [3]. In an autonomous driving scenario, this means that the model can better integrate surrounding information (such as data from multiple cameras, lidar, etc.), leading to a more comprehensive understanding of the driving environment.

B. Transformers in Deep RL

Transformers have rapidly gained traction in RL research as a way to address limitations of CNNs and RNNs. The self-attention mechanism in transformers allows modeling of long-range dependencies and has been proposed to handle partial observability and complex input spaces in RL. For instance, the Decision Transformer treats RL as a sequence modeling problem, using a transformer to output actions from past trajectory data, and has achieved promising results on Atari and robotic tasks by leveraging temporal context [4]. In online RL, Parisotto et al. introduced the ViT for RL to replace CNN encoders; while requiring large data, these approaches show that RL agents can benefit from global attention. A recent survey by Agarwal et al. concludes that transformers improve performance in RL across domains like robotics and games by tackling unstable training and long-horizon credit assignment issues [5].

C. Transformers for Autonomous Driving

Within autonomous driving, researchers have begun to explore transformer-based architectures to improve decision-making. Liu et al. proposed a Scene-Rep Transformer that learns a latent representation of the driving scene (encoding interactions among vehicles and their intended routes) which is then fed into an SAC agent [6]. This method yielded substantial gains in data efficiency and driving performance (higher success rates and safer behavior) in dense urban simulations. Ge et al. developed a DRL framework (DRLNDT) that augments SAC with a transformer to model temporal dependencies in the observation history [7]. Their approach improved policy robustness to occlusions and sensor noise, outperforming a recurrent SAC baseline in success rate and cumulative reward. These works validate that transformers can enhance RL decision-makers by providing better representations of either the spatial scene or temporal context. Autonomous driving demands not only high performance but also reliability and safety. The Transformer-based model, thanks to its powerful representation capabilities, tends to produce more stable and robust policies that can adapt to complex and dynamic driving scenarios. Our work differs in that we use the CNN model to processing visual state input, output the image to a transformer encoder, focusing on improving value function estimation in SAC.

III. METHODOLOGY

A. Soft Actor-Critic (SAC)

SAC is an off-policy actor-critic algorithm that optimizes a stochastic policy for continuous actions while maximizing entropy to encourage exploration [8]. It maintains two function

struggle with sample inefficiency in high-dimensional visual environments. The Transformer architecture excels at capturing long-range dependencies and fusing multi-modal approximators: an Actor (policy $\pi_\phi(a|s)$ which outputs actions), and a Critic (Q-network $Q_\theta(s,a)$ which evaluates state-action pairs). SAC typically uses twin critics $Q_{\theta_1}, Q_{\theta_2}$ with independent CNN/MLP networks to mitigate overestimation bias (the ‘‘clipped double-Q’’ trick). The critics are trained to minimize the Bellman error:

$$\mathcal{L}_Q = \mathbb{E} \left[\left(Q_\theta(s,a) - \left(r + \gamma(1 - \text{done}) \left(\min_{Q_{\text{target}}} (Q(s',a')) - \alpha \log \pi(a'|s') \right) \right) \right)^2 \right] \quad (1)$$

Where r is the immediate reward, γ the discount factor for future rewards, and done flags episode termination. Q_{target} uses twin networks' minimum to curb overestimation, α balances exploration via entropy (linked to $\log_\pi(a'|s')$, the next action's log-probability).

SAC stands out with its entropy-regularized objective, which provides robust exploration and smoother policy outputs in continuous action spaces. This is particularly valuable for autonomous driving, where subtle adjustments matter. Both SAC and DQN are off-policy and can leverage replay buffers for improved sample efficiency. PPO offers very stable policy updates through its clipping mechanism, though it may require more samples due to its on-policy nature. For autonomous driving tasks that require fine-grained continuous control and robust exploration, SAC often offers significant advantages over DQN and even PPO, particularly in terms of smoothness and sample efficiency.

In practice, SAC alternates between environment interaction (collecting transitions in a replay buffer) and gradient updates to θ, ϕ using mini-batches. For high-dimensional observations like images, the actor and critic networks include convolutional encoders to extract state features. In our baseline CNN-SAC implementation, both actor and critic use a CNN to process input images (e.g., four convolutional layers with ReLU, producing a latent state feature vector). This baseline mirrors prior works and serves as the point of comparison for our proposed method.

B. Transformer-Based SAC Architecture

We modify the SAC actor and critic networks by replacing its CNN network with a transformer encoder while keeping other components unchanged. Figure 1 illustrates the architecture of the proposed Transformer-based SAC architecture. Instead of splitting the image into patches and embed each patch into a vector, we first feeding an image observation through convolutional neural network for dimension reduction. The CNN output is send to embedding for transformer blocks. Physical positional encoding are added to retain spatial information. This sequence of patch embeddings is input to a multi-layer Transformer Encoder that applies self-attention to model relationships among all patches (hence all parts of the scene). The transformer encoder outputs a sequence of transformed embeddings, including an updated [CLS] token that serves as a global representation of the scene.

We then concatenate this [CLS] embedding (a fixed-length vector) with the action input (which may be a continuous steering/throttle vector) and pass them through a small Multi-Layer Perceptron (MLP head) to predict . In our design, the

Transformer-based SAC produces one action and two Q-values (for the twin critics) by using transformer encoders with separate MLP heads.

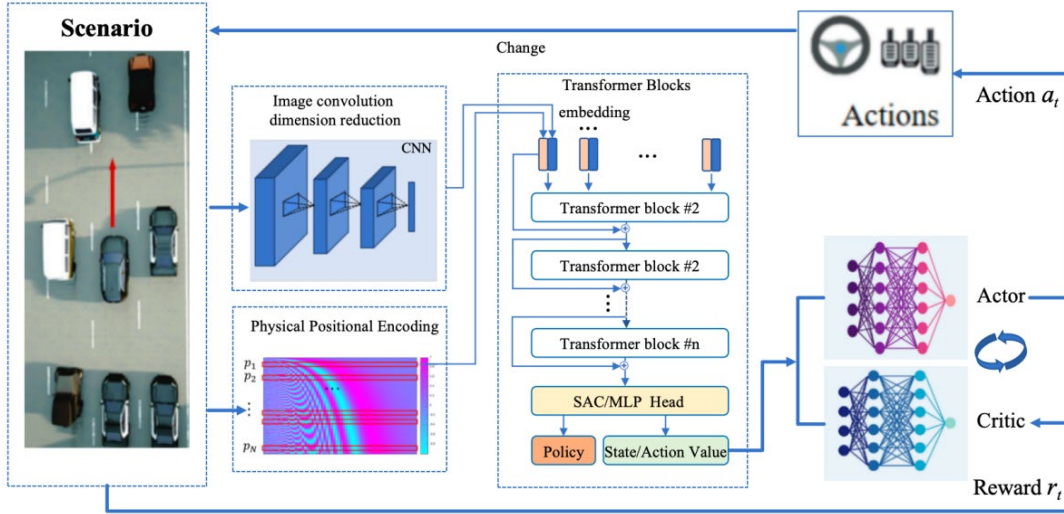


Figure 1. Transformer-based SAC architecture. The image observation is divided into patches by a CNN network and passed through a Transformer Encoder, plus physical position encoding, producing a global scene representation. This is concatenated with the action vector and fed into an MLP head to output the action and Q-values.

$$PPE_{2k}(\cdot, pos_{ph}) = \sin\left(\frac{pos_{ph}}{(2N_{pos})^{\frac{2k}{D}}}\right) \quad (2)$$

$$PPE_{2k+1}(\cdot, pos_{ph}) = \cos\left(\frac{pos_{ph}}{(2N_{pos})^{\frac{2k}{D}}}\right) \quad (3)$$

Physical Position Encoding: The Transformer was initially designed for Natural Language Processing (NLP) research, where the order of words is of paramount importance. The original Transformer architecture introduced the classic method of positional encoding using alternating sine and cosine functions. In the context of autonomous driving tasks, spatial information is naturally of great significance. Although positional information is embedded within the vehicle state inputs, it can be diluted by other information such as velocity, intent, and mapping data. The superposition of physical position encoding after the feature extraction step can enhance positional features and improve network performance. In this paper, we simply refer to the original Transformer and generate the Physical Position Encoding (PPE) in the form of a combination of sine and cosine functions. The map within the region of interest is discretized into N_{pos} physical positions. For the physical position at $pos_{ph} - 1$, the PPE is calculated according to Equation 2 and Equation 3, where $2k$ and $2k + 1$ are index of PE vector, D is the model dimension.

C. Advantages of the Transformer based SAC architecture

The key intuition is that self-attention enables the SAC to consider contextual dependencies in the scene that a CNN might miss. For instance, if a pedestrian is about to jaywalk far ahead of the ego-car, a CNN might focus on nearby vehicles and not integrate that distant pedestrian feature until late convolutional layers. The transformer, in contrast, can directly attend to the pedestrian patch and the ego-vehicle's patch

simultaneously in one layer, effectively modeling the interaction (“if the pedestrian moves, the car’s optimal action changes”). This global receptive field and interaction modeling is especially beneficial in autonomous driving, where the relative positions and intentions of multiple agents determine the reward (collision or safe pass). Prior work has shown transformers can encode interactions and intentions among agents in traffic, so we leverage that capacity here. Another benefit is the ability to integrate multi-modal inputs: our transformer design could easily incorporate non-visual tokens (e.g., a route plan or speed indicator as additional input tokens), making the critic more aware of the driving context, whereas a CNN is specialized for images. In summary, the Transformer-based SAC is expected to produce more accurate Q-value estimates by globally understanding the scene, which in turn should lead to a better policy update for the actor and improved overall performance.

D. Training Setup and Hyperparameters

We implemented the Transformer-based SAC using PyTorch, building on the standard SAC algorithm. The environment for training and evaluation is the CARLA simulator (version 0.9.10), set up with urban driving scenarios such as unprotected left turns, merges, and roundabouts with other vehicles and pedestrians. The observation space consists of a front-view RGB camera image (128×128 pixels) and some low-dimensional state info (speed, etc.). The image is processed by the CNN and the transformer as described; the low-dimensional state is concatenated to the [CLS] embedding before the MLP head. For fair comparison, both CNN-SAC (baseline) and Transformer-SAC use the same training hyperparameters unless otherwise noted. We utilized a replay buffer of 1e6 transitions and trained with a batch size of 256. The discount factor was $\gamma = 0.99$. Both actor and critic

optimizers were Adam with learning rate 3×10^{-4} . To stabilize transformer training, we employed layer normalization and initialized the transformer weights from a normal distribution (no pre-training was used; the transformer learned from scratch during RL training). The transformer encoder in the critic had 4 layers, 8 attention heads, and a hidden dimension of 256. This yields a comparable number of parameters to the CNN encoder it replaced. We found it important to tune the entropy temperature in α SAC: we used the automatic entropy tuning method from Haarnoja et al. so that the policy entropy is adjusted during training. Each training run lasted for 2 million environment steps (which corresponds to roughly 50 hours of simulated driving). We conducted 5 runs with different random seeds for each algorithm variant. Training was performed on a single GPU; the transformer-based critics did increase computation time per step by about 20%, a reasonable overhead given the improvements we will discuss

IV. EXPERIMENTS AND RESULTS

A. Simulation Environment

We evaluated the algorithms on several challenging driving scenarios in CARLA. Specifically, we used a benchmark scenarios: Roundabout (navigate a multi-lane roundabout with other vehicles). This scenario test the agent's ability to make safe decisions under dynamic and partially observable conditions. A success in an episode is defined as reaching the goal (exiting the intersection, completing the turn, etc.) without any collision or traffic rule violation; failure occurs if the vehicle crashes or times out. We used the same reward structure for both SAC variants: a sparse success reward at completion, small step penalties to encourage efficiency, and collision penalty of -1. In the CARLA simulation environment, the autonomous driving agent dynamically interacts with the simulation environment through three primary control commands: acceleration, steering, and braking. The value ranges of these three parameters are all floating-point numbers. Specifically, the range of acceleration is $[0, 1]$, the range of steering is $[-1, 1]$, and the range of braking is $[0, 1]$. The Soft Actor-Critic (SAC) algorithm is known as a continuous deep reinforcement learning algorithm, which requires the selection of actions in a continuous manner. Therefore, at each time step, the agent's action a_t is composed of a triplet (acceleration_t, steering_t, brake_t), with each parameter's range defined as above. Utilizing the built-in traffic participant dynamic generator in CARLA, a mixed traffic scenario comprising pedestrians, four-wheeled vehicles, two-wheeled motor vehicles, and non-motorized vehicles is constructed. The spatial distribution strategy employed is based on Gaussian mixture distribution sampling according to the road network topology.

B. Baseline vs. Transformer Performance

We compare our Transformer-based SAC agent to the baseline CNN-Critic SAC agent. Figure 2 shows representative training curves of average episode return, and Table 1 summarizes the quantitative results. We observed that the Transformer-based SAC consistently learns faster and achieves

higher asymptotic performance than the CNN-based SAC. For example, the CNN-SAC baseline often struggled with entering the roundabout without collisions, plateauing at around 89-91% success rate. In contrast, the Transformer-SAC learned a more effective merging strategy, reaching over 95% success.

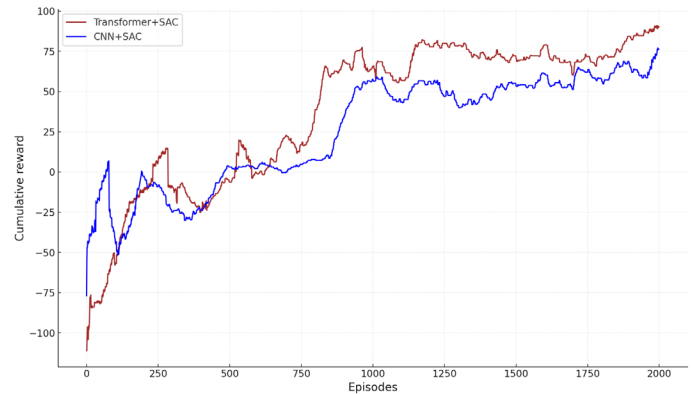


Figure 2. Training curves of average episode return compare between Transformer-based SAC and CNN-based SAC.

TABLE I. TRANSFORMER-BASED SAC AND CNN-BASED SAC ALGORITHM PERFORMANCE QUANTITATIVE EVALUATION

Model	AV number	Success rate (%)	Collision rate (%)	Average return
Transformer Based SAC	140	99	2	91.87
	210	95	5	85.16
CNN Based SAC	140	91	8	85.16
	210	89	12	74.29
Transformer Based SAC without PPE	140	95	4	89.76
	210	92	8	81.23

Transformer-based SAC had an average success rate improvement of about 8% over the baseline. Notably, Transformer-SAC also converged in fewer steps, this indicates better sample efficiency, likely due to the more informative state representations learned by the transformer. In terms of reward, the final average episode return for Transformer-SAC was higher in the scenario. As an illustrative case, in the pedestrian crossing scenario, Transformer-SAC achieved an average return of 91 (out of a maximum 100) whereas CNN-SAC got around 82—because the transformer agent learned to slow down early for crossing pedestrians, avoiding penalty, whereas the CNN agent occasionally failed to notice the pedestrian until too late. We also measured safety metrics: the collision rate per episode for Transformer-SAC was roughly half of that of the baseline (e.g., 5% vs 10% in intersection scenario). This suggests that the transformer-augmented critic better evaluates risky states, enabling the policy to proactively avoid accidents. To demonstrate the significance of physical positional encoding in autonomous driving behavior decision-making models, we conducted ablation experiments in Table 1. Without incorporating the physical position encoding module, the overall performance of the behavior decision-making model

deteriorated significantly, however, the performance is still superior to that of the SAC algorithm based on CNN.

V. CONCLUSION

In this paper, we presented an improved SAC algorithm for autonomous driving by replacing the CNN module with a Transformer-based module. Through our experiments in realistic driving simulation scenarios, we showed that this change leads to faster learning and significantly better decision-making performance. The Transformer-based SAC achieved higher success rates in navigating complex traffic interactions and demonstrated more foresighted driving behavior than the conventional CNN-based SAC. These results underline the importance of rich scene representations in reinforcement learning for robotics: by capturing global context and interactions, the transformer enables the agent to make safer and more effective decisions.

Our findings contribute to the growing body of research integrating transformers into reinforcement learning, especially in safety-critical domains like autonomous driving. The key takeaway is that state representation learning is a critical bottleneck in deep RL for autonomous driving, and advanced architectures like transformers can considerably alleviate this bottleneck by providing the policy with a deeper understanding of its environment. We believe this is a step toward more reliable RL-driven autonomous vehicles.

Investigating pre-training methods for the vision transformer in the driving context could improve sample efficiency and help transfer policies from simulation to reality (sim-to-real gap). Another promising avenue is to extend the approach to handle sequential decision-making under partial

observability, for example by using transformers that attend over time steps (capturing temporal context in addition to spatial context). We also plan to test the approach on real-world datasets or in scaled-down real robot cars to verify that the gains observed in simulation carry over to physical systems. Overall, the introduction of transformers into the SAC algorithm shows great potential for advancing autonomous driving decision intelligence.

REFERENCES

- [1] B. B. Elallid et al., "Enhancing Autonomous Driving Navigation Using Soft Actor-Critic," *Future Internet*, vol. 16, no. 7, p. 238, 2024.
- [2] M. Savari and Y. Choe, "Online Virtual Training in Soft Actor-Critic for Autonomous Driving," 2021 International Joint Conference on Neural Networks (IJCNN), Shenzhen, China, pp. 1-8, 2021, doi: 10.1109/IJCNN52387.2021.9533791.
- [3] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention is all you need," In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*, Red Hook, NY, USA, pp. 6000–6010, 2017.
- [4] L. Chen, K. Lu, A. Rajeswaran, et al., "Decision Transformer: Reinforcement Learning via Sequence Modeling," *arXiv preprint arXiv:2106.01345*, 2021.
- [5] P. Agarwal et al., "Transformers in Reinforcement Learning: A Survey," *arXiv:2307.05979*, 2023.
- [6] H. Liu et al., "Augmenting Reinforcement Learning with Transformer-based Scene Representation for Autonomous Driving," *IEEE Trans. Neural Netw. Learn. Syst.*, 2023.
- [7] L. Ge et al., "Deep RL Navigation via Decision Transformer in Autonomous Driving," *Front. Neurobotics*, vol. 18, 2024.
- [8] T. Haarnoja et al., "Soft Actor-Critic: Off-Policy Maximum Entropy Deep RL with a Stochastic Actor," *arXiv preprint arXiv:1801.01290*, 2018.